

Google Summer of Code, 2026

Proposal for Checkstyle



Project: [OpenJdk Code Convention Coverage](#)

Name: Aman Baliyan

GitHub: [Aman-Baliyan](#)

LinkedIn: [Aman Baliyan](#)

Email: baliyanaman200@gmail.com

Time Zone: Indian Standard Time (IST), UTC +5:30

Project Size: Large

Mentors: Roman Ivanov, Mauryan Kansara, Baratali Izmailov, Daniel Mühlbachler

TABLE OF CONTENTS

ABOUT ME	3
PROJECT DETAILS	4
HOW STYLE GUIDES ARE IMPLEMENTED?	5
UPDATING OLD OPENJDK STYLE GUIDE	6
REFRACTING TESTS	12
UPDATING NEW OPENJDK STYLE GUIDE	12
PROJECT TIMELINE	26
EXPECTATIONS FROM MENTORS	28
CONTRIBUTIONS HIGHLIGHTS	28
WHY ME?	31
FUTURE WORK	31

ABOUT ME

Greetings! I'm Aman Baliyan, currently pursuing my second year in MCA Computer Engineering at National Institute Of Technology, Jamshedpur, India. Over the past two years, I've immersed myself in the world of coding, cultivating a strong proficiency in Java, C++, JavaScript, and Spring boot. My strong desire to explore and continuously learn motivates me to constantly discover new fields and technologies.

My Open-Source journey began in December 2025 when I started contributing to Checkstyle. Through this experience, I've gained invaluable insights into the practical application of Java in real-world projects. I gained knowledge about mutation testing, implementing new checks in checkstyle, maven plugin usage, code coverage, test case development, etc. I also got to learn about visitor design patterns and its utilization in actual projects.

Contributing to Checkstyle has not only made me better at coding, but I also got to meet some amazing folks of the community and got to learn so much from them.

I enjoy contributing to Checkstyle because it is a beginner friendly organization, well documented and it is also very active. Maintainers are always ready to help contributors to solve issues and fix bugs.

PROJECT DETAILS

Checkstyle currently provides partial support for the [OpenJDK Code Convention](#) through the [Sun configuration](#), which was created in 2004. Since then, many new modules and validation rules have been added to Checkstyle, making the existing configuration outdated and incomplete. Although the OpenJDK Code Convention is marked as outdated, its best practices are still widely used across projects, so accurate and complete coverage remains important.

In addition, the new [OpenJDK Java Style Guidelines](#) are close to becoming the successor to the original convention. This project focuses on systematically reviewing both the old and new guidelines, ensuring that all rules are supported in Checkstyle. Missing functionality will be implemented, existing bugs will be fixed, and coverage documentation will be updated to clearly reflect the level of support for each rule.

The main aim of this project is to provide complete and up-to-date support for OpenJDK coding conventions in Checkstyle, making these well-known coding practices easily verifiable, improving overall code quality, and enhancing the user experience for developers.

Project Deliverables:

- Review and update OpenJDK Code Convention configuration.
- Implement missing checks for both old and new guidelines.
- Fix reported and blocking issues in existing modules.
- Create and update coverage documentation pages.

The project details can be also found at [wiki-page](#).

HOW STYLE GUIDES ARE IMPLEMENTED?

We use xml files to implement Checkstyle checks based on our requirements. Similarly, OpenJdk style guide is implemented in the [sun_checks.xml](#) configuration file. It contains different modules/checks and suppressions which are used to enforce different rules of the style guide. The Configuration page contains all the information about how Checkstyle's checks can be implemented with the help of an xml file.

The style guide uses various Checkstyle's checks to ensure all the rules of style guide are being followed properly. Different checks have different functionalities, like Javadoc Comments Checks ensure all the Javadoc rules are being followed, similarly Naming Checks ensure that the variables and classes are properly named, etc. The coverage table section of the Style Guide's Coverage page contains all the checks used to implement the style guide.

Sometimes a single rule in the style guide does not always map 1:1 with Checkstyle's check. So to overcome this issue, we use suppressions to suppress or modify some functionality or property of the Check so it can follow style guide's rule. Here are all the suppressions mentioned which are used in the implementation of the openjdk style guide: [link](#)

We use a combination of different suppressions & checks to enforce the rules of style guides.

UPDATING OLD OPENJDK STYLE GUIDE

The `sun_checks.xml` was created in 2004. After that, new modules are added and changed in checkstyle. The project aims to update the style guide implementation and increase coverage of guidelines based on new modules.

There were many sections which are not covered in the style guide, below I've listed all the sections which needs updates or needs to be implemented:

- 3.1 - Java Source Files
- 4 - Indentation
- 4.2 - Wrapping Lines
- 5.1.1 - Block Comments
- 5.1.2 - Single Line Comments
- 6.2 - Initialization
- 6.4 - Classes and Interface Declarations
- 7.3 - Return statements
- 7.4 to 7.9 - if , if - else, for , while, do - while, switch statements
- 9 - Naming Conventions

All the rules mentioned above are explained here:

3.1 - Java Source Files

Each Java source file contains a single public class or interface. When private classes and interfaces are associated with a public class, you can put them in the same source file as the public class. The public class should be the first class or interface in the file.

Needs to discuss on this rule will this rule can be fulfilled by [OneTopLevelClass](#) module or it requires a new module.

4 - Indentation

It requires 4 to be used as the unit of indentation. The current config file does not follow this rule we can enforce this by using the [Indentation](#) module.

4.2 - Wrapping Lines

It requires that when a line is broken, the break should occur **after** a comma and **before** an operator.

We have support for "Break before an operator" using the [Operator wrap](#) module with the `n1` (newline) option. We also have support for "Break after a comma" using the [Separator wrap](#) module with the `eo1` (end of line) option.

However, the current `sun_checks.xml` does not include the `SeparatorWrap` configuration for commas. I will update the configuration to include this module. Additionally, for the "8-space rule" mentioned in this section, I am configuring the [Indentation](#) module's `lineWrappingIndentation` property to 8 to fulfill this specific requirement.

5.1.1 - Block Comments & 5.1.2 - Single Line Comments

It requires that **block and single-line comments** be indented to the same level as the code they describe and always be **preceded by a blank line**.

We have support for matching comment indentation via the [CommentsIndentation](#) module. We should discuss the “**blank line before comments**” rule. The existing [EmptyLineSeparator](#) check already enforces spacing before classes and methods, where comments are usually placed. This means the rule is partially handled implicitly. So, do we really need a separate check for comments, or would that be redundant?

6.4 - Classes and Interface Declarations

Missing rules for this section are **methods be separated by a blank line** and that **null statements** (empty methods) have the closing brace `}` appear immediately after the opening brace `{`

We have support for line separator via the [EmptyLineSeparator](#) module and for the second missing rule i am working on new module here is the link of that issue: [link](#)

7.3 - return Statements

A `return` statement with a value should not use parentheses unless they make the return value more obvious in some way. Example:

 Copy

```
return;return myDisk.size();  
return (size ? size : defaultSize);
```

We have [UnnecessaryParentheses](#) module for this just need to add the config in xml file

7.4 to 7.9 if, if - else, while, for, do - while, switch statements

Format of control statements:

```
if (condition) {statements;
}
if (condition) {
statements;
} else {
statements;
}
if (condition) {
statements;
```

```
do {statements;
} while (condition);
```

```
try {statements;
} catch (ExceptionClass e) {
statements;
}
```

We already have support for the formatting of these control statements via [LeftCurly](#) and [RightCurly](#) but their configuration in sun_checks.xml is not written as clearly or precisely as in the google_checks.xml. Hence there is a need for change in their configuration.

8.1 - Blank Lines

Two blank lines should always be used in the following circumstances:

- Between sections of a source file
- Between class and interface definitions

One blank line should always be used in the following circumstances:

- Between methods
- Between the local variables in a method and its first statement
- Before a block (see section 5.1.1) or single-line (see section 5.1.2) comment
- Between logical sections inside a method to improve readability

We have an [EmptyLineSeparator](#) module for a single line but according to this rule we need 2 empty lines . I think we need a new check here for this.

9 - Naming Conventions

Already have support for this chapter just need to change the default value of [MethodName](#) and [TypeName](#) modules to follow the rule given below:

Except for variables, all instance, class, and class constants are in mixed case with a lowercase first letter. Internal words start with capital letters. Variable names should not start with underscore _ or dollar sign \$ characters, even though both are allowed.

REFRACTING TESTS

All the tests for openjdk style guide implementation are located in this [directory](#).

Refactoring of openjdk style guide tests includes transitioning from per module testing to chapter wise testing as written for the google_checks.xml. Excluding chapter 3 in this [directory](#) all chapters used per module testing.

Also some sections are covered in config file but they did not have any test related to them .

UPDATING NEW OPENJDK STYLE GUIDE

Currently the openjdk config has support for only a section of [New OpenJDK Java Style Guidelines](#). So we have to add support for each section. Here are the sections which are present in the new openjdk guidelines:

- [Java Source Files](#)
- [Formatting](#)
- [Naming](#)
- [Programming Practices](#)

More details about the sections are given below along with the modules required to fulfill the rules:

2 - Java Source Files

Source files must be encoded in 7-bit ASCII.

All lines must be terminated with a line feed character (LF, ASCII value 10) and *not* for instance CR or CR+LF.

There may be no trailing white space at the end of a line.

The name of a source file must equal the name of the class it contains followed by the .java extension, even for files that only contain a package private class. This does not apply to files that do not contain any class declarations, such as package-info.java.

- **Encoding:** I will set the `charset` property of the `Checker` module to `US-ASCII`.
- **Line Endings:** I will use the `NewlineAtEndOfFile` module with the `lineSeparator` property set to `lf`.
- **Filename:** I will use the `OuterTypeFilename` module to ensure the class name and filename are synchronized.

2.1 - Special Characters

It requires that Space (ASCII 32) is the only allowed whitespace character besides the line feed. It strictly forbids the use of actual Tab characters (even in strings) and requires that common escapes like `\t`, `\n`, and `\r` be used instead of their hex or octal equivalents.

- **No Tabs:** i will use the [FileTabCharacter](#) module in the Checker to ban all tab characters globally.
- **Unicode Escapes:** I will use the [AvoidEscapedUnicodeCharacters](#) module to restrict the use of Unicode escapes for characters that have printable equivalents.

3 - Formatting

There may be only one top level class declaration per file.

This rule can be enforced by using [OneTopLevelClass](#) module

3.2 - Package Declaration

It requires that the package declaration should not be line wrapped even if it exceeds the maximum length limit.

Can be applied by using [NoLineWrap](#) module.

3.3 - Import Statements

It requires that **import statements** are sorted first by type (non-static before static), then by specific package origin (**java**, **javax**, external, then internal), and finally lexicographically. It also forbids line wrapping for imports and the presence of any unused imports.

- **Import Ordering:** I will configure the [ImportOrder](#) module with the `groups` property set to `java`, `javax` followed by external/internal patterns, and set `staticGroups` to the end to ensure non-static imports appear first.
- **No Unused Imports:** I will include the [UnusedImports](#) module to satisfy the requirement for clean import lists.
- **No Line Wrapping:** I will use the [NoLineWrap](#) module for this.

3.3.1 - Wildcard Imports

In any case, no more than one wildcard import per file should be used.

We have the [AvoidStarImport](#) module but this module does not have any property that allows the count of star imports . So there are two ways we can do that:

- By adding a new field in existing module
- Create a new module

We need to discuss this.

3.4 - Class Structure

The recommended order of class members is the following:

1. Fields
2. Constructors
3. Factory methods
4. Other Methods

Nested types are put at the top of the class or right before it is first used.

Order for class structure is covered by [DeclarationOrder](#) but in checkstyle we can not determine between factory method and normal static method so this rule is partially covered.

For ordering of Nested type we have to build a new module named **NestedTypeFirst**.

3.4.1 - Order Of Constructors and Overloaded Methods

It requires constructors and overloaded methods to be grouped together and sorted according to the increasing arity.

We have [OverloadedMethodsDeclarationOrder](#) for overloaded methods and [ConstructorsDeclarationGrouping](#) for constructors grouping but we do not have support for sorting according to increasing arity.

3.5 - Modifiers

Modifiers should go in the following order

1. Access modifier (`public` / `private` / `protected`)
2. `abstract`
3. `static`
4. `final`
5. `transient`
6. `volatile`
7. `default`
8. `synchronized`
9. `native`
10. `strictfp`

Modifiers should not be written out when they are implicit.

Modifiers order can be covered by **ModifierOrder** but some changes need to be made in this module to fully cover the order and **RedundantModifier** is used to fulfill the requirement that implicit modifiers are not used.

I am not sure about whether to use **FinalLocalVariable** or not using the because of this rule:

Method parameters and local variables should not be declared final unless it improves readability or documents an actual design decision.

3.6 - Braces

It requires that opening braces appear at the end of the current line (**EOL**) and that keywords like `else`, `catch`, and `while` (in do-while) appear on the **same line** as the preceding closing brace. It also mandates the use of braces even for single-line blocks to avoid bugs, while allowing "Short Forms" for trivial, single-line methods or enums.

We have support for these rules using the `LeftCurly`, `RightCurly`, and `NeedBraces` modules. I will configure `LeftCurly` with `option="eol"`, `RightCurly` with `option="same"`, and `NeedBraces` to ensure all control structures are enclosed.

There is a confusion in this rule -

There should be a new line in front of a closing brace unless the block is empty (see *Short Forms* below)

Here it says only for empty blocks right and left curly should be on same line but in examples it has :

Dos

```
enum Response { YES, NO, MAYBE }
```

```
public boolean isReference() { return true; }
```

I am working on `rightCurlyAloneOrEmpty` to implement the empty block rule.

3.7 - Indentation

Indentation level is four spaces.

Only space characters may be used for indentation. No tabs.

Empty lines must not be indented. (This is implied by the no trailing white space rule.)

case lines should be indented with four spaces, and statements within the case should be indented with another four spaces.

- **Indentation Level:** I will use the `Indentation` module for this with `offset` 4.
- **No Tabs And Empty Indentation:** Covered in Section 2
- **Case Line Ident:** Need to set `caseIdent` property to 4.

3.8 - Wrapping Lines

Source code and comments should generally not exceed 80 characters per line and rarely if ever exceed 100 characters per line, including indentation.

Can be enforced by using the `LineLength` module but we have to set the fixed length 80 or 100.

3.8.1 - Wrapping Class Declaration

A class header should not be wrapped unless it approaches the maximum column limit.

If it does, it may be wrapped before `extends` and/or `implements` keywords.

Declarations of type parameters may, if necessary, be wrapped the same way as method arguments

To fulfill the requirements for class declarations we can use `NoLineWrap` and `Indentation` modules.

3.8.2 - Wrapping Method Declaration

Method declarations can be formatted by listing the arguments vertically, or by a new line and +8 extra spaces

If a throws clause needs to be wrapped, put the line break in front of the throws clause and make sure it stands out from the argument list, either by indenting +8 relative to the function declaration, or +8 relative to the previous line.

To fulfill the requirements for **Method Declarations**, I will use the `Indentation` module with `lineWrappingIndentation` and `throwsIndent` both set to `8`.

3.8.3 - Wrapping Expressions

Break before operators.

Break before the `.` in chained method calls.

To fulfill the requirements for **Expression Wrapping**, I will use the `OperatorWrap` and `SeparatorWrap` modules with the `n1` option.

3.9 - WhiteSpace

3.9.1 - Vertical WhiteSpace

A single blank line should be used to separate...

- Copyright notice
- Package declaration
- Class declarations
- Constructors
- Methods
- Static initializers
- Instance initializers

...and may be used to separate logical groups of

- import statements
- fields
- statements

These rules can be enforced by using the `SeparatorWrap` module.

3.9.2 - Horizontal WhiteSpace

A single space should be used...

- To separate keywords from neighboring opening or closing brackets and braces
- Before and after all binary operators and operator like symbols such as arrows in lambda expressions and the colon in enhanced for loops (but not before the colon of a label)
- After // that starts a comment.
- After commas separating arguments and semicolons separating the parts of a for loop.
- After the closing parenthesis of a cast.

For rule 1 and 2 `WhiteSpaceAround` is used and for rule 3 `RegexpSingleLineJava` is used and for rule 4 and 5 `WhiteSpaceAfter` is used.

3.10 - Variable Declarations

It requires one variable per declaration and array declarations like this (`String[] args`) and variable declaration nearest to its first use.

These rules can be enforced by these modules:

- `MultipleVariableDeclarations`
- `ArrayTypeStyle`
- `VariableDeclarationUsageDistance`

3.11 - Annotations

It requires whether all annotations are put on the same line or on different lines for a declaration being annotated.

We can not configure for both rules at the same time according to current checkstyle modules either there is a need to create a new module or add a new property in existing module

`AnnotationLocation`.

3.12 - Lambda Expressions

Expression lambdas are preferred over single-line block lambdas.

The parameter types should be omitted unless they improve readability.

If a lambda expression stretches over more than a few lines, consider creating a method.

We have an `LambdaBodyLength` module to check the length of lambda but checkstyle does not have any support for the two rules mentioned above. Implementing these would require further discussion and alignment with the maintainers.

3.13 - Redundant Parenthesis

Redundant grouping parentheses (i.e. parentheses that does not affect evaluation) may be used if they improve readability.

Redundant grouping parentheses should typically be left out in shorter expressions involving common operators but included in longer expressions or expressions involving operators whose precedence and associativity is unclear without parentheses. Ternary expressions with non-trivial conditions belong to the latter.

The entire expression following a return keyword must not be surrounded by parentheses.

Needs clarification on this section whether to use `UnnecessaryParenthesis` for tokens other than `return`.

3.14 - Literals

long literals should use the upper case letter `L` suffix.

Hexadecimal literals should use upper case letters `A-F`.

All other numerical prefixes, infixes, and suffixes should use lowercase letters.

We have `UpperEll`, `NumericalPrefixesInfixesAndSuffixes`, `HexLiteralCase` modules for these rules.

3.15 - Javadoc

Start longer comments with a short summarizing sentence since Javadoc includes this in the method summary table.

Prefer inline tags (such as `{@code ...}` and `{@link ...}` etc) over corresponding HTML tags (`<code>...</code>`, `<a href="...">...</a>` etc).

Use `<p>` to separate paragraphs (closing `</p>` tags are not needed and should not be used)

We can enforce these rules using `SummaryJavadoc`, `JavadocStyle` and `JavadocParagraph`.

4 - Naming

Avoid hiding/shadowing methods, variables and type variables in outer scopes.

We have `HiddenField` for this.

4.1 - Package Names

Package names should be all lower case without underscores or other special characters.

We have `PackageName` for this

4.2 - Class, Interface and Enum Names

Use mixed case with the first letter in each word in upper case.

We have `TypeName` for this.

4.3 - Method Names & 4.4 Variable

It requires the use of a mix case with the first letter in lowercase. We have `MethodName`, `LocalVariableName`, `ParameterName`, `MemberName` for this.

4.5 - Type Variables

Use single capital letters or for longer names use capital letters with underscore.

We have [`ClassTypeParameterName`](#), [`MethodTypeParameterName`](#), [`InterfaceTypeParameterName`](#).

5 - Programming Practices

Always use `@Override` where it is possible to do so.

Document any non-obvious pre and post conditions (also for private methods). In particular, all parameters / return values are assumed to be non-null unless explicitly stated otherwise.

Avoid checking in “TODO” comments. If the missing code doesn’t affect correctness or performance, skip the comment, otherwise file an issue for it.

Avoid checking in dead code.

Static mutable state should be used judiciously. In particular enums should not have mutable state, and any public static variable must also be final.

Make fields final unless they have a need to be mutable.

Qualify static method invocations and fields with class identifiers and not variable identifiers.

Fall through on non-empty cases in switch statements should be documented, typically with a “// fall through” comment.

Mutating method parameters is discouraged. This is because parameters are typically associated with the input values and by mutating these the code may be harder to understand, especially when only looking at parts of the method below the mutation.

Keep methods short. Consider splitting up long methods into smaller ones to limit variable scopes and simplify structure of the code.

It requires that no todo comments are used and make public static variables should be final and fall through on non - empty cases in switch should be documented and methods should be small and make fields final if they need no change and method parameters should not be mutated.

- **Todo Comments:** We have a [TodoComments](#) module for this.
- **Static Variable Constraints:** We have a [VisibilityModifier](#) module to satisfy the requirement.
- **Final Fields:** We have [FinalLocalVariable](#) and [FinalParameter](#) modules for this.
- **Switch Fall-Through:** We have a [FallThrough](#) module for this.
- **Switch Fall-Through:** For method length we have a [MethodLength](#) module.

PROJECT TIMELINE

Weeks	Start date	End date	Tasks to be completed
-	May 1	May 24	Community Bonding period
1	May 25	June 2	Adding the missing modules in sun_checks.xml along with tests
2	June 3	June 10	Adding support for rightCurlyAloneOrEmpty in both old and new openjdk configs. Here is the issue link
3	June 11	June 20	Implementing a new module for section 8.1 of old openjdk guidelines and adding support for it in the style guide implementation.
4	June 21	June 24	Works on section 2 to section 3.3 adding their support in new openjdk.xml also add test cases for them
5	June 25	June 30	Working on section 3.3.1 add new feature for Star imports. Here is the issue link .
6	June 31	July 7	Adding a new module for section 3.4 .
7 - 8	July 7	July 21	Works on section 3.4.1 add new features in two existing modules to support openjdk rules.

9	July 22	August 4	Works on section 3.5 to 5 add required modules in configs along with tests.
10	August 4	August 10	Works on section 3.11 and 3.12
11	August 11	August 20	Solve issues related to other checks and review them to check if they need new functionalities.
12	August 21	August 31	The project will be reviewed and the final reports will be submitted to the reviewers.

Different sections require varying amounts of time to complete, so I have allocated days to each section based on their complexity and requirements.

I will be dedicating around 35–40 hours per week to ensure steady progress and timely completion of the project. I have a dedicated two-month academic break during which I can fully commit my time and focus to this project.

During the community bonding period, I will actively discuss with the maintainers the sections where I have any confusion or lack clarity in requirements. I will also explore and discuss potential new functionalities that can be implemented beyond my current research, ensuring that the project scope is both complete and improved where possible.

I will also begin contributing early by submitting initial PRs to gather feedback and align with project expectations.

Given the scope of the project, I aim to complete all planned work within the deadline. However, if any unforeseen circumstances arise, I will communicate proactively with the maintainers and adjust accordingly to ensure the quality and completeness of the project.

EXPECTATIONS FROM MENTORS

This project requires implementing new modules and functionalities, in which I will need to have maintainers opinions and get help from them if I get stuck over something. I need maintainers to take time for reviewing the new functionalities or modules and give valuable feedback so I can complete my tasks in an efficient way.

CONTRIBUTIONS HIGHLIGHTS

Sr	Pull Requests	Status
New modules & bug fixes		
1	Issue #16678: new check for NumericalCharacterCase	Merged
2	Issue #18653: Update HexLiteralCaseCheck add hex float literals	Merged
3	Issue #18486: Refractor JavadocParagraph Check	Merged
Pitest Issues And New Validation Tests		

4	Issue #18025: Resolve pitest Suppression Tree walker	Merged
5	Issue #18303: make method public and add test	Merged
6	Issue #19064: Validation of number of test in xdocs	Merged
Checkstyle-openrewrite-recipes		
8	Issue #138: add new recipe for NumericalPrefixesInfixesSuffixesCase	Merged
9	Issue #144: Update HexLiteral for hex Float	Merged
10	Issue #147: Remove redundant verify method	Merged
11	Issue #158: Add recipe for MissingOverride	Reviewing
12	Issue #139: Upgrade to jdk 21	Merged
XPath Regression Testing		
10	Issue #19064: Add third test in XpathRegressionWhitespaceAfterTest	Merged
11	Issue #19064: Add third Test For TypeName	Merged
12	Issue #19064: Add third Missing Test	Merged
Resolve Error-prone		
13	Issue #18599: Resolve error-prone PatternMatchingInstanceof	Merged
14	Issue #17988: Resolve error-prone JdkObsolete	Merged
15	Issue #17988: Resolve error-prone	Merged

Here all my pr's are listed for [checkstyle](#) and [checkstyle-openrewrite-recipe](#)

Issues i opened:

- [new feature required to allow atleast one star import in a file](#)
- [False-negative: HexLiteralCase does not throw violation on Hex float literals](#)

WHY ME?

I have been actively contributing to Checkstyle since December 2025, with 20+ merged PRs that demonstrate my consistent involvement and strong engagement with the project. Over the past three months of hands-on contributions, I have developed a solid understanding of Checkstyle's architecture, coding standards, and development workflow.

My contributions include implementing new modules, fixing bugs, and improving existing functionality. Notably, I have worked on increasing the coverage of the literal section by introducing a new module and resolving issues in existing modules. This experience has allowed me to work with some of the more complex parts of the codebase and understand how rules are designed, validated, and maintained.

With a strong technical background in Java, static code analysis, and software development, I am comfortable making impactful changes while maintaining high code quality. I have demonstrated solid problem-solving skills through debugging complex issues and writing clean, efficient code. Combined with my familiarity with the project and proven contribution track record, I am well-prepared to contribute effectively and deliver reliable results for this project.

FUTURE WORK

By contributing to checkstyle, I've gained precious experience of how Java works in real world projects, gained beneficial skills and knowledge of Continuous Integration, Mutation Testing, Code Coverage, Open-Source and community work.

After GSoC I will continue my contribution to Checkstyle, I will make quality PRs, help newcomers and solve their doubts. I've gained important skills by contributing to Checkstyle and I wish to continue expanding my knowledge even further.

I've thought of some new features ideas to add in Checkstyle, I would love to discuss them with the maintainers and implement them and improves checkstyle user experience.